

# Chapitre 1

## Implémentation informatique

Ce chapitre présente l'implémentation informatique du modèle de Cox-Ross-Rubinstein. Le projet est développé en Python et organisé en plusieurs modules indépendants, chacun ayant un rôle précis dans la chaîne de traitement allant de la récupération des données de marché au calcul du prix de l'option.

### 1.1 Architecture du projet

L'architecture du projet suit le principe de séparation des concepts. Chaque module ne fait qu'une chose, puis les dépendances s'organisent de la donnée brute vers le résultat final.

L'organisation des fichiers sources est la suivante :

- `option.py` : Définition de la classe `Option`.
- `market_data.py` : Téléchargement et préparation des données de marché.
- `calibration.py` : Estimation de la volatilité et calibration des paramètres CRR.
- `crr.py` : Construction de l'arbre binomial et calcul du prix.
- `backtester.py` : Évaluation du modèle sur plusieurs dates.

Le flux de données suit la chaîne logique suivante :

`market_data.py`  $\longrightarrow$  `calibration.py`  $\longrightarrow$  `crr.py`  $\longrightarrow$  `backtester.py`

Le module `option.py` est transversal : il est utilisé par `crr.py` et `backtester.py` pour représenter l'option à valoriser. Les notebooks Jupyter orchestrent l'ensemble en important ces modules et en produisant les résultats numériques et graphiques du projet.

### 1.2 Classe Option

La classe `Option`, définie dans le fichier `option.py`, représente une option financière par ses trois paramètres contractuels fondamentaux : le strike, la maturité et le type (call ou put).

#### 1.2.1 Attributs

Le constructeur `__init__` reçoit trois arguments :

- `strike` : le prix d'exercice  $K$ , doit être strictement positif ;
- `maturity` : la maturité  $T$  en années, doit être strictement positive ;
- `option_type` : une chaîne de caractères valant `'call'` ou `'put'`.

Ces trois valeurs sont stockées comme attributs de l'objet après vérification.

### 1.2.2 Méthode de calcul du payoff

La méthode `payoff(S)` calcule le gain obtenu à maturité pour un prix de sous-jacent  $S_T = S$  donné. Elle implémente directement les formules théoriques :

$$\text{payoff}(S) = \begin{cases} \max(S - K, 0) & \text{si l'option est un call,} \\ \max(K - S, 0) & \text{si l'option est un put.} \end{cases}$$

Cette méthode est appelée par `crr.py` lors du calcul des payoffs terminaux de l'arbre (feuilles de l'arbre) ainsi que lors de la comparaison avec la valeur de continuation pour les options américaines.

### 1.2.3 Vérification des paramètres

Dès l'instanciation, le constructeur lève une exception `ValueError` dans les cas suivants :

- `strike ≤ 0` : un prix d'exercice nul ou négatif est économiquement incohérent ;
- `maturity ≤ 0` : une maturité nulle ou négative n'a pas de sens ;
- `option_type` différent de `'call'` ou `'put'` : toute autre valeur serait une erreur de saisie.

Ces vérifications garantissent que tout objet `Option` valide peut être utilisé sans précaution supplémentaire dans les modules en aval.

## 1.3 Module `market_data`

Le module `market_data.py` est responsable de la récupération et de la préparation des données de marché. Il utilise la bibliothèque `yfinance` pour télécharger les données de cours et `pandas` pour les manipuler.

### 1.3.1 Téléchargement des données

La fonction `download_data(ticker, start_date, end_date)` télécharge les données OHLCV (Open, High, Low, Close, Volume) d'un actif financier identifié par son symbole boursier (par exemple `^FCHI` pour le CAC 40 ou `BZ=F` pour le Brent). Les données sont triées par ordre chronologique croissant et renvoyées sous forme de `DataFrame`.

La fonction principale d'entrée du module est `build_dataset`, qui enchaîne toutes les étapes de traitement et retourne un `DataFrame` enrichi avec les variables dérivées nécessaires au projet :

- `Close` : Prix de clôture journalier.
- `log_return` : Rendement logarithmique journalier.
- `cum_return` : Rendement cumulé en base 100.
- `rolling_vol_21` : Volatilité glissante annualisée sur 21 jours.

### 1.3.2 Nettoyage des données

La fonction `clean_data` effectue plusieurs opérations de nettoyage indispensables à la fiabilité des calculs :

- suppression des colonnes multi-niveaux éventuelles (`MultiIndex`) produites par `yfinance` ;
- conversion de l'index en format `datetime` ;
- suppression des doublons de dates (conservation de la première occurrence) ;
- tri chronologique ascendant ;
- suppression des lignes contenant au moins une valeur manquante (`NaN`).

### 1.3.3 Calcul des rendements

La fonction `compute_log_returns` calcule les rendements logarithmiques journaliers à partir de la série de prix de clôture :

$$r_t = \ln\left(\frac{S_t}{S_{t-1}}\right).$$

Ces rendements sont utilisés par `calibration.py` pour estimer la volatilité historique. La dernière valeur de clôture disponible  $S_0$  et la série complète des log-rendements sont extraites via la fonction `get_S0_and_log_returns`, qui constitue l'interface entre `market_data.py` et les modules de calibration et de pricing.

## 1.4 Module calibration

Le module `calibration.py` estime les paramètres nécessaires à la construction de l'arbre binomial à partir des données de marché. Il ne fait pas de pricing : son rôle est uniquement de fournir les paramètres  $\sigma$ ,  $r$ ,  $\Delta t$ ,  $u$ ,  $d$  et  $p^*$  à `crr.py`.

### 1.4.1 Estimation de la volatilité

La fonction `estimate_volatility` estime la volatilité historique annualisée à partir de la série des rendements logarithmiques :

$$\hat{\sigma} = \text{std}(r_1, \dots, r_n) \times \sqrt{252},$$

où l'écart-type est calculé avec la correction de Bessel (`ddof` = 1), ce qui donne un estimateur sans biais de la variance d'un échantillon. Le facteur 252 correspond au nombre conventionnel de jours de cotation par an.

Le module propose également une fonction `volatility_confidence_interval` qui construit un intervalle de confiance bilatéral pour  $\hat{\sigma}$  à partir de la loi du  $\chi^2$  à  $n - 1$  degrés de liberté, conformément à l'hypothèse de normalité des log-rendements.

### 1.4.2 Gestion du taux sans risque

La fonction `get_risk_free_rate` fournit le taux sans risque  $r$  utilisé dans le modèle. Si aucun taux n'est passé en argument, la valeur par défaut est  $r = 2\%$  par an, ce qui correspond à un niveau raisonnable pour un marché européen. La fonction vérifie que le taux est positif ou nul et lève une exception dans le cas contraire.

### 1.4.3 Calcul des paramètres du modèle

Les fonctions `compute_time_step`, `compute_ud` et `compute_risk_neutral_probability` calculent respectivement :

$$\begin{aligned}\Delta t &= \frac{T}{N}, \\ u &= e^{\sigma\sqrt{\Delta t}}, \quad d = \frac{1}{u} = e^{-\sigma\sqrt{\Delta t}}, \\ p^* &= \frac{e^{r\Delta t} - d}{u - d}.\end{aligned}$$

La probabilité risque-neutre  $p^*$  est vérifiée : elle doit être comprise strictement entre 0 et 1 pour garantir l'absence d'opportunité d'arbitrage.

La fonction `calibrate_crr_parameters` enchaîne toutes ces étapes en un seul appel et retourne un dictionnaire contenant les six paramètres  $\{\sigma, r, \Delta t, u, d, p^*\}$ , directement exploitable par `crr.py` et les notebooks.

## 1.5 Module `crr`

Le module `crr.py` implémente le cœur du modèle de Cox-Ross-Rubinstein : la construction de l'arbre binomial des prix du sous-jacent, le calcul des payoffs terminaux et la rétropropagation permettant d'obtenir le prix de l'option.

### 1.5.1 Construction de l'arbre binomial

La fonction `build_price_tree(S_0, u, d, n_steps)` construit l'arbre des prix du sous-jacent. L'arbre est représenté comme une liste de listes : le niveau  $i$  contient les  $i + 1$  nœuds de l'étape  $i$ , où le nœud  $j$  correspond au prix :

$$S_{i,j} = S_0 \cdot u^j \cdot d^{i-j}, \quad 0 \leq j \leq i.$$

Cette représentation est efficace en mémoire car elle exploite la propriété de recombinaison de l'arbre CRR : un nœud atteint après une hausse puis une baisse est identique à un nœud atteint après une baisse puis une hausse.

### 1.5.2 Calcul des payoffs terminaux

La fonction `compute_terminal_payoffs` parcourt les nœuds du dernier niveau de l'arbre (les feuilles) et calcule le payoff de l'option en chacun d'eux en appelant la méthode `payoff` de l'objet `Option`.

### 1.5.3 Pricing européen

La fonction `backward_induction` effectue la rétropropagation pour une option européenne. En partant des payoffs terminaux, elle remonte l'arbre niveau par niveau en appliquant à chaque nœud  $(i, j)$  la formule d'actualisation risque-neutre :

$$V_{i,j} = e^{-r \Delta t} [p^* \cdot V_{i+1,j+1} + (1 - p^*) \cdot V_{i+1,j}].$$

Le prix de l'option est la valeur au nœud racine  $V_{0,0}$ .

### 1.5.4 Pricing américain

La fonction `backward_induction_american` adapte la rétropropagation au cas américain. À chaque nœud intermédiaire, la valeur de l'option est le maximum entre la valeur de continuation (actualisation risque-neutre) et la valeur d'exercice immédiat :

$$V_{i,j} = \max(e^{-r \Delta t} [p^* V_{i+1,j+1} + (1 - p^*) V_{i+1,j}], \text{payoff}(S_{i,j})).$$

Cette comparaison à chaque nœud constitue la différence fondamentale entre le pricing européen et le pricing américain.

La fonction principale `crr_price` regroupe toutes ces étapes : elle vérifie la validité des paramètres d'entrée, détermine le nombre d'étapes  $N = \text{round}(T/\Delta t)$ , construit l'arbre des prix, puis appelle la procédure de rétropropagation appropriée selon que l'option est européenne ou américaine.

## 1.6 Module backtester

Le module `backtester.py` évalue le modèle CRR de manière empirique en répétant le calcul du prix théorique sur une longue série de dates. Son rôle est d'orchestrer l'évaluation, et non de refaire la logique de pricing.

### 1.6.1 Fonctionnement du backtest

La fonction `run_backtest` parcourt la série temporelle des données de marché à partir d'une date de départ déterminée par la taille de la fenêtre de calibration (`window = 252` jours par défaut). Pour chaque date  $t$  :

1. on extrait les 252 rendements logarithmiques précédents (fenêtre glissante) ;
2. on calibre les paramètres  $\sigma$ ,  $u$ ,  $d$ ,  $p^*$  sur cette fenêtre ;
3. on calcule le prix théorique de l'option avec `crr_price` ;
4. le résultat est enregistré avec la date, le prix du sous-jacent et les paramètres calibrés.

Cette approche par fenêtre glissante permet d'observer comment les paramètres et les prix évoluent dans le temps, et de tester la stabilité du modèle face aux différents régimes de marché.

### 1.6.2 Calcul des erreurs

Le module fournit des fonctions de calcul d'erreurs utilisables dès lors que des prix de référence sont disponibles. La fonction `compare_prices` construit un tableau de comparaison ligne à ligne entre prix prédits et prix observés, avec l'erreur absolue et l'erreur relative pour chaque date. Les fonctions `compute_mae` et `compute_rmse` calculent respectivement :

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |\hat{V}_t - V_t|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (\hat{V}_t - V_t)^2}.$$

### 1.6.3 Métriques d'évaluation

La fonction `summarize_backtest_results` produit un résumé synthétique du backtest. En l'absence de prix d'options observés sur le marché, le résumé contient les statistiques descriptives des prix prédits (moyenne, minimum, maximum), ainsi que les moyennes de la volatilité calibrée  $\hat{\sigma}$  et de la probabilité risque-neutre  $p^*$  sur toute la période.

La fonction `analyze_stability` complète cette analyse en calculant des statistiques glissantes sur les prix prédits (moyenne et écart-type sur une fenêtre de 21 jours), ce qui permet de visualiser les régimes de marché au sein de la série temporelle.

## Chapitre 2

# Expérimentations numériques

Ce chapitre présente les résultats numériques obtenus en appliquant le modèle de Cox-Ross-Rubinstein aux données réelles. L'objectif est de vérifier la cohérence du modèle, d'en illustrer le fonctionnement sur des exemples concrets et d'analyser l'influence des paramètres sur le prix calculé.

### 2.1 Paramètres choisis

#### 2.1.1 Choix du sous-jacent

Le sous-jacent retenu est l'indice boursier CAC 40, identifié par le symbole  $\text{^FCHI}$  sur la plateforme Yahoo Finance. La période d'observation couvre les données journalières du 1<sup>er</sup> janvier 2018 au 1<sup>er</sup> juin 2026. Cette fenêtre temporelle est suffisamment longue pour inclure des épisodes de volatilité très différents : une phase de hausse régulière avant 2020, le choc de la crise Covid en mars 2020 avec une baisse brutale de l'indice et un pic de volatilité annualisée dépassant 60 %, puis une période de reprise et de relative stabilité en 2021–2022.

#### 2.1.2 Choix du strike

L'option est définie *at-the-money* : le prix d'exercice  $K$  est fixé égal au dernier prix de clôture disponible de l'indice, soit  $K = S_0$ . Ce choix est classique dans les études pédagogiques car il permet de se concentrer sur la dynamique du modèle sans introduire un biais lié à la moneyness de l'option.

#### 2.1.3 Choix de la maturité

La maturité est fixée à  $T = 1$  an. Ce choix est cohérent avec une volatilité exprimée en base annuelle et permet une comparaison directe entre call et put sans distorsion due à l'horizon temporel. Des maturités alternatives (3 mois, 6 mois, 2 ans) sont examinées dans l'analyse de sensibilité.

#### 2.1.4 Choix du nombre d'étapes

Le nombre d'étapes de l'arbre est fixé à  $N = 100$ . Ce choix résulte d'un compromis entre précision numérique et temps de calcul. L'analyse de convergence présentée en section 2.6 montre qu'à partir de  $N = 100$ , les variations du prix sont inférieures à quelques centimes, ce qui est suffisant pour les besoins du projet.

Le taux sans risque est fixé à  $r = 2\%$  par an, une valeur raisonnable pour un marché européen sur la période considérée.

## 2.2 Exemple d'arbre binomial

Pour illustrer concrètement le fonctionnement du modèle, on construit un arbre binomial sur un petit exemple à  $N = 3$  étapes. À partir du prix initial  $S_0$  et des facteurs de hausse  $u$  et de baisse  $d$  calibrés sur le CAC 40, l'arbre des prix du sous-jacent est :

| Étape   | Nœud 0    | Nœud 1  | Nœud 2    | Nœud 3    |
|---------|-----------|---------|-----------|-----------|
| $i = 0$ | $S_0$     |         |           |           |
| $i = 1$ | $S_0 d$   | $S_0 u$ |           |           |
| $i = 2$ | $S_0 d^2$ | $S_0$   | $S_0 u^2$ |           |
| $i = 3$ | $S_0 d^3$ | $S_0 d$ | $S_0 u d$ | $S_0 u^3$ |

On constate la propriété de recombinaison : le nœud central à  $i = 2$  vaut  $S_0 \cdot u \cdot d = S_0$  (car  $d = 1/u$ ), indépendamment de l'ordre dans lequel la hausse et la baisse se produisent. Cette propriété réduit le nombre de nœuds distincts de  $2^N$  à  $\frac{N(N+1)}{2} + 1$ , ce qui rend le calcul efficace même pour des valeurs élevées de  $N$ .

Les payoffs terminaux d'un call de strike  $K = S_0$  sont alors calculés pour chacun des quatre nœuds de la dernière étape. Les nœuds inférieurs au strike donnent un payoff nul, et seuls les nœuds en hausse génèrent un gain. La rétropropagation remonte ensuite ces valeurs vers la racine.

## 2.3 Pricing d'un call européen

On valorise un call européen *at-the-money* sur le CAC 40, avec  $T = 1$  an,  $N = 100$  étapes et  $r = 2\%$ . La volatilité historique annualisée est estimée sur l'ensemble de la période et vaut  $\hat{\sigma}$ , ce qui conduit aux paramètres calibrés suivants :

$$u = e^{\hat{\sigma}\sqrt{\Delta t}}, \quad d = \frac{1}{u}, \quad p^* = \frac{e^{r\Delta t} - d}{u - d}.$$

La condition d'absence d'arbitrage est vérifiée :  $0 < p^* < 1$ .

La fonction `crr_price` est appelée avec le paramètre `american=False`. La rétropropagation européenne applique à chaque nœud la formule d'actualisation risque-neutre et retourne le prix au nœud racine. Ce prix représente la prime théorique que l'acheteur doit payer pour acquérir le droit d'acheter l'indice au prix  $K = S_0$  dans un an.

L'interprétation économique est la suivante : plus la volatilité est élevée, plus la dispersion des prix terminaux est grande, et plus le call a de valeur car les gains en cas de hausse sont illimités tandis que les pertes sont bornées par zéro.

## 2.4 Pricing d'un put européen

On valorise un put européen sur le même actif, avec les mêmes paramètres. Un nouvel objet `Option` est instancié avec `option_type='put'`. Le payoff terminal de chaque nœud est désormais  $\max(K - S_{N,j}, 0)$ .

La rétropropagation est identique à celle du call européen : seule la définition du payoff change. Le prix obtenu est la prime théorique que l'acheteur du put doit payer pour le droit de vendre l'indice au prix  $K = S_0$  dans un an.

On vérifie la parité call-put, relation fondamentale de la théorie des options européennes :

$$C - P = S_0 - K \cdot e^{-rT}.$$

Pour une option *at-the-money* ( $K = S_0$ ), cette relation se réduit à :

$$C - P = S_0 (1 - e^{-rT}) > 0,$$

ce qui implique que le call européen vaut légèrement plus que le put européen de mêmes caractéristiques lorsque  $r > 0$ . Les résultats numériques obtenus sont bien conformes à cette relation.

## 2.5 Pricing d'une option américaine

On valorise à présent un put américain sur le CAC 40, avec les mêmes paramètres. La seule modification dans l'implémentation est le passage du paramètre `american=True` à la fonction `crr_price`, qui appelle alors `backward_induction_american`.

À chaque nœud intermédiaire, la valeur de l'option est le maximum entre la valeur de continuation actualisée et la valeur d'exercice immédiat :

$$V_{i,j} = \max(e^{-r\Delta t} [p^* V_{i+1,j+1} + (1 - p^*) V_{i+1,j}], K - S_{i,j}).$$

Le prix obtenu pour le put américain est supérieur ou égal à celui du put européen de mêmes caractéristiques. Cet écart, appelé *prime d'exercice anticipé*, traduit la valeur de la flexibilité supplémentaire offerte par le contrat américain : il peut être optimal d'exercer le put avant maturité lorsque le sous-jacent est suffisamment bas.

En revanche, pour un call américain sur un actif ne versant pas de dividendes, il n'est jamais optimal d'exercer avant maturité, et le prix du call américain est donc identique à celui du call européen. Ce résultat théorique est confirmé par les calculs numériques.

## 2.6 Influence du nombre d'étapes

On étudie la convergence du prix du call européen en fonction du nombre d'étapes  $N$ . Les valeurs  $N \in \{25, 50, 100, 200, 500\}$  sont testées, tous les autres paramètres étant maintenus constants.

Les résultats montrent une convergence rapide et monotone du prix vers une valeur limite. À partir de  $N = 100$ , les variations de prix entre deux valeurs consécutives de  $N$  sont inférieures à quelques centimes, ce qui confirme que ce choix est suffisant pour les besoins du projet.

Ce comportement est conforme au résultat théorique établi par Cox, Ross et Rubinstein : lorsque  $N \rightarrow +\infty$ , le prix CRR converge vers le prix de Black-Scholes. L'arbre binomial constitue ainsi une approximation discrète de la dynamique continue log-normale, et le raffinement du maillage améliore la précision de cette approximation.

En pratique, il existe un compromis entre précision et temps de calcul : doubler  $N$  divise par deux la taille du pas de temps  $\Delta t$  et multiplie approximativement par quatre le nombre de nœuds de l'arbre. Le choix  $N = 100$  offre une précision satisfaisante avec un temps d'exécution de l'ordre de la milliseconde.